Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки

Лабораторна робота №3

з дисципліни
«ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ »

на тему :

«Розробка інтерфейсу користувача»

Виконала:

Студентка групи ІМ-54

Дрига О.О

Номер в списку: 11

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:

1. Створити у середовищі розробки проект з ім’ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно з варіантом (Ж = 12):

● Масив об'єктів (12 mod 3 = 0):
Використовується динамічний масив вказівників Shape **pcshape. Кількість елементів масиву $N = 12 + 100 = 112$.

● "Гумовий" слід при вводі (12 mod 4 = 0):
Суцільна лінія чорного кольору.

● Увід та відображення прямокутника:
○ Спосіб вводу (12 mod 2 = 0): По двом протилежним кутам.
○ Відображення (12 mod 5 = 2): Чорний контур з кольоровим заповненням.
○ Колір заповнення (12 mod 6 = 0): Жовтий.

● Увід та відображення еліпсу:
○ Спосіб вводу (12 mod 2 = 0): Від центру до одного з кутів охоплюючого прямокутника.
○ Відображення (12 mod 5 = 2): Чорний контур еліпсу без заповнення.

● Позначка поточного об’єкту (12 mod 2 = 0):
Відображається в меню (метод OnInitMenuPopup).

● Інтерфейсні та архітектурні вимоги:
○ Наявність меню "Об’єкти" (з підпунктами українською мовою: Крапка, Лінія, Прямокутник, Еліпс), яке розташоване між меню "Файл" та "Довідка".
○ Наявність панелі інструментів Toolbar з кнопками-іконками та спливаючими підказками (tooltips), які дублюють підпункти меню "Об’єкти".
○ Усі методи-обробники повідомлень, зокрема метод OnNotify, повинні бути функціями-членами відповідних класів.
○ У звіті обов'язково має бути представлена схема успадкування класів (діаграма класів).

Вихідний текст головного файлу:

main.cpp

```
#define NOMINMAX
```

```
#include <windows.h>

#include <windowsx.h>

#include <algorithm>

#include <cmath>

#include "Shapes.h"

#include "Toolbar.h"



class MainWindow {

    static const int N = 112;



    HWND    hWnd;

    HINSTANCE hInst;

    HMENU   hObjectsMenu;

    Toolbar toolbar;



    Shape** pcshape;

    int     shapeCount;

    int     currentObjectType;

    bool    isDrawing;

    POINT   ptStart, ptCurrent;
```

```
    void DrawRubberTrace(POINT p1, POINT p2) {

        HDC hdc = GetDC(hWnd);

        SetROP2(hdc, R2_NOTXORPEN);



        HPEN   hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HPEN   hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc,
GetStockObject(NULL_BRUSH));



        if (currentObjectType == IDM_RECT) {

            Rectangle(hdc, std::min(p1.x, p2.x), std::min(p1.y, p2.y),

                           std::max(p1.x, p2.x), std::max(p1.y, p2.y));

        }

        else if (currentObjectType == IDM_ELLIPSE) {

            long dx = std::abs(p2.x - p1.x);

            long dy = std::abs(p2.y - p1.y);

            Ellipse(hdc, p1.x - dx, p1.y - dy, p1.x + dx, p1.y + dy);

        }

        else {

            MoveToEx(hdc, p1.x, p1.y, NULL);

            LineTo(hdc, p2.x, p2.y);
```

```
        }


        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

        ReleaseDC(hWnd, hdc);

    }


    HMENU CreateAppMenu() {

        HMENU hMenu = CreateMenu();

        HMENU hFileMenu = CreatePopupMenu();

        hObjectsMenu = CreatePopupMenu();

        HMENU hHelpMenu = CreatePopupMenu();


        AppendMenuW(hFileMenu, MF_STRING, IDM_EXIT, L"Вихід");


        AppendMenuW(hObjectsMenu, MF_STRING, IDM_POINT, L"Крапка");

        AppendMenuW(hObjectsMenu, MF_STRING, IDM_LINE, L"Лінія");

        AppendMenuW(hObjectsMenu, MF_STRING, IDM_RECT, L"Прямокутник");

        AppendMenuW(hObjectsMenu, MF_STRING, IDM_ELLIPSE, L"Еліпс");
```

```
        AppendMenuW(hHelpMenu, MF_STRING, IDM_ABOUT, L"Про програму");



        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");

        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hObjectsMenu,
L"Об'єкти");

        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Довідка");

        return hMenu;

    }



    void AddShape(Shape* s, long x1, long y1, long x2, long y2) {

        if (shapeCount < N) {

            s->Set(x1, y1, x2, y2);

            pcshape[shapeCount++] = s;

            InvalidateRect(hWnd, NULL, TRUE);

        }

        else {

            delete s;

        }

    }



    LRESULT OnCreate() {
```

```
    SetMenu(hWnd, CreateAppMenu());
    toolbar.Create(hWnd, hInst);
    return 0;
}

LRESULT OnSize() {
    toolbar.AutoSize();
    return 0;
}

LRESULT OnCommand(WPARAM wParam) {
    int id = LOWORD(wParam);
    switch (id) {
    case IDM_POINT:
    case IDM_LINE:
    case IDM_RECT:
    case IDM_ELLIPSE:
        currentObjectType = id;
        toolbar.Check(id);
        break;
    case IDM_ABOUT:
```

```
            MessageBoxW(hWnd,
                L"Лабораторна робота №3\nВаріант №12\nВиконала:
студентка Дрига Олександра ІМ-54",
                L"Про програму", MB_OK | MB_ICONINFORMATION);
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;
        }
        return 0;
    }


    LRESULT OnNotify(LPARAM lParam) {
        LPNMHDR hdr = (LPNMHDR)lParam;
        if (hdr->code == TTN_GETDISPINFOW) {
            LPNMTTDISPINFOW tip = (LPNMTTDISPINFOW)lParam;
            tip->lpszText = (LPWSTR)Toolbar::TipText((int)hdr->idFrom);
        }
        return 0;
    }
```

```
    LRESULT OnInitMenuPopup(WPARAM wParam, LPARAM lParam) {

        HMENU hMenu = (HMENU)wParam;

        if (HIWORD(lParam) == 0 && hMenu == hObjectsMenu) {

            CheckMenuRadioItem(hMenu, IDM_POINT, IDM_ELLIPSE,

                               currentObjectType, MF_BYCOMMAND);

        }

        return 0;

    }



    LRESULT OnLButtonDown(LPARAM lParam) {

        ptStart.x = GET_X_LPARAM(lParam);

        ptStart.y = GET_Y_LPARAM(lParam);

        ptCurrent = ptStart;



        if (currentObjectType == IDM_POINT) {

            AddShape(new PointShape(), ptStart.x, ptStart.y, ptStart.x,
ptStart.y);

            isDrawing = false;

        }

        else {

            isDrawing = true;
```

```
        SetCapture(hWnd);
        DrawRubberTrace(ptStart, ptCurrent);
    }
    return 0;
}

LRESULT OnMouseMove(LPARAM lParam) {
    if (isDrawing) {
        DrawRubberTrace(ptStart, ptCurrent);
        ptCurrent.x = GET_X_LPARAM(lParam);
        ptCurrent.y = GET_Y_LPARAM(lParam);
        DrawRubberTrace(ptStart, ptCurrent);
    }
    return 0;
}

LRESULT OnLButtonUp(LPARAM lParam) {
    if (isDrawing) {
        DrawRubberTrace(ptStart, ptCurrent);
        isDrawing = false;
        ReleaseCapture();
```

```
            ptCurrent.x = GET_X_LPARAM(lParam);
            ptCurrent.y = GET_Y_LPARAM(lParam);

            Shape* s = nullptr;
            switch (currentObjectType) {
            case IDM_LINE:    s = new LineShape();      break;
            case IDM_RECT:    s = new RectangleShape(); break;
            case IDM_ELLIPSE: s = new EllipseShape();   break;
            }
            if (s) AddShape(s, ptStart.x, ptStart.y, ptCurrent.x,
ptCurrent.y);
        }
        return 0;
    }

    LRESULT OnPaint() {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        for (int i = 0; i < shapeCount; i++)
            if (pcshape[i]) pcshape[i]->Show(hdc);
```

```
    EndPaint(hWnd, &ps);
    return 0;
}

LRESULT OnDestroy() {
    PostQuitMessage(0);
    return 0;
}

LRESULT HandleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:         return OnCreate();
    case WM_SIZE:           return OnSize();
    case WM_COMMAND:        return OnCommand(wParam);
    case WM_NOTIFY:         return OnNotify(lParam);
    case WM_INITMENUPOPUP:  return OnInitMenuPopup(wParam, lParam);
    case WM_LBUTTONDOWN:    return OnLButtonDown(lParam);
    case WM_MOUSEMOVE:      return OnMouseMove(lParam);
    case WM_LBUTTONUP:      return OnLButtonUp(lParam);
    case WM_PAINT:          return OnPaint();
    case WM_DESTROY:        return OnDestroy();
```

```
        }

        return DefWindowProcW(hWnd, msg, wParam, lParam);

    }



    static LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wParam,
LPARAM lParam) {

        MainWindow* self = (MainWindow*)GetWindowLongPtrW(hWnd,
GWLP_USERDATA);

        if (msg == WM_NCCREATE) {

            self =
(MainWindow*)((LPCREATESTRUCTW)lParam)->lpCreateParams;

            self->hWnd = hWnd;

            SetWindowLongPtrW(hWnd, GWLP_USERDATA, (LONG_PTR)self);

        }

        if (self) return self->HandleMessage(msg, wParam, lParam);

        return DefWindowProcW(hWnd, msg, wParam, lParam);

    }



public:

    MainWindow(HINSTANCE h)

        : hWnd(NULL), hInst(h), hObjectsMenu(NULL), shapeCount(0),

          currentObjectType(IDM_POINT), isDrawing(false) {
```

```
    pcshape = new Shape * [N];
    for (int i = 0; i < N; i++) pcshape[i] = nullptr;
    ptStart.x = ptStart.y = ptCurrent.x = ptCurrent.y = 0;
}

~MainWindow() {
    for (int i = 0; i < shapeCount; i++) delete pcshape[i];
    delete[] pcshape;
}

bool Create(int nCmdShow) {
    const wchar_t cls[] = L"Lab3WindowClass";

    WNDCLASSEXW wc = { 0 };
    wc.cbSize = sizeof(wc);
    wc.style = CS_HREDRAW | CS_VREDRAW;
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInst;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszClassName = cls;
```

```
    if (!RegisterClassExW(&wc)) return false;

    HWND h = CreateWindowExW(0, cls, L"Lab3 - Графічний редактор",
        WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN,
        CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
        NULL, NULL, hInst, this);
    if (!h) return false;

    ShowWindow(h, nCmdShow);
    UpdateWindow(h);
    return true;
}

int Run() {
    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}
```

```
};



int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int nCmdShow)
{

    MainWindow app(hInstance);

    if (!app.Create(nCmdShow)) return 0;

    return app.Run();

}
```

shape.h

```
#ifndef SHAPE_H

#define SHAPE_H



#ifndef NOMINMAX

#define NOMINMAX

#endif

#include <windows.h>

#include <algorithm>

#include <cstdlib>



class Shape {

protected:
```

```
    long xs1, ys1, xs2, ys2;

public:

    Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}

    virtual ~Shape() {}



    virtual void Set(long x1, long y1, long x2, long y2) {

        xs1 = x1; ys1 = y1;

        xs2 = x2; ys2 = y2;

    }



    virtual void Show(HDC hdc) = 0;

};



#endif
```

shapes.h

```
#ifndef SHAPES_H

#define SHAPES_H



#include "Shape.h"



class PointShape : public Shape {
```

```
public:
    void Show(HDC hdc) override {
        SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));
        MoveToEx(hdc, xs1 - 1, ys1, NULL); LineTo(hdc, xs1 + 2, ys1);
        MoveToEx(hdc, xs1, ys1 - 1, NULL); LineTo(hdc, xs1, ys1 + 2);
    }
};


class LineShape : public Shape {
public:
    void Show(HDC hdc) override {
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        MoveToEx(hdc, xs1, ys1, NULL);
        LineTo(hdc, xs2, ys2);

        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);
    }
};
```

```
class RectangleShape : public Shape {
public:
    void Show(HDC hdc) override {
        long left   = std::min(xs1, xs2);
        long top    = std::min(ys1, ys2);
        long right  = std::max(xs1, xs2);
        long bottom = std::max(ys1, ys2);

        HPEN   hPen   = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 0));

        HPEN   hOldPen   = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Rectangle(hdc, left, top, right, bottom);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        DeleteObject(hBrush);
```

```
    }
};

class EllipseShape : public Shape {
public:
    void Show(HDC hdc) override {
        long dx = std::abs(xs2 - xs1);
        long dy = std::abs(ys2 - ys1);

        HPEN   hPen      = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HPEN   hOldPen   = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc,
GetStockObject(NULL_BRUSH));

        Ellipse(hdc, xs1 - dx, ys1 - dy, xs1 + dx, ys1 + dy);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
    }
};
```

```
#endif
```

toolbar.h

```
#ifndef TOOLBAR_H

#define TOOLBAR_H



#ifndef _WIN32_IE

#define _WIN32_IE 0x0501

#endif



#include <windows.h>

#include <commctrl.h>

#include "Shape.h"



#ifndef BTNS_CHECKGROUP

#define BTNS_CHECKGROUP 0x0006

#endif



#ifdef _MSC_VER

#pragma comment(lib, "comctl32.lib")

#endif
```

```
#define IDM_POINT    1001
#define IDM_LINE     1002
#define IDM_RECT     1003
#define IDM_ELLIPSE  1004
#define IDM_EXIT     1005
#define IDM_ABOUT    1006
#define ID_TOOLBAR   2001

class Toolbar {
    HWND       hwnd;
    HIMAGELIST hImages;

    static const int ICON = 24;
    static const int COUNT = 4;

    void DrawIcons() {
        HDC scr = GetDC(NULL);
        HDC mem = CreateCompatibleDC(scr);
        HBITMAP bmp = CreateCompatibleBitmap(scr, ICON * COUNT, ICON);
        HBITMAP oldBmp = (HBITMAP)SelectObject(mem, bmp);
```

```
        // фон = маска (маджента)
        RECT all = { 0, 0, ICON * COUNT, ICON };
        HBRUSH bg = CreateSolidBrush(RGB(255, 0, 255));
        FillRect(mem, &all, bg);
        DeleteObject(bg);

        HPEN   pen1 = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HPEN   pen2 = CreatePen(PS_SOLID, 2, RGB(0, 0, 0));
        HBRUSH yellow = CreateSolidBrush(RGB(255, 255, 0));
        HBRUSH black = CreateSolidBrush(RGB(0, 0, 0));
        HPEN   oldPen = (HPEN)SelectObject(mem, pen1);
        HBRUSH oldBrush = (HBRUSH)SelectObject(mem,
GetStockObject(NULL_BRUSH));

        int ox;

        ox = 0;
        MoveToEx(mem, ox + 12, 4, NULL);  LineTo(mem, ox + 12, 21);
        MoveToEx(mem, ox + 4, 12, NULL);  LineTo(mem, ox + 21, 12);
        SelectObject(mem, black);
```

```
        Ellipse(mem, ox + 9, 9, ox + 16, 16);



        ox = ICON;

        SelectObject(mem, pen2);

        MoveToEx(mem, ox + 4, 20, NULL);  LineTo(mem, ox + 20, 4);



        ox = ICON * 2;

        SelectObject(mem, pen1);

        SelectObject(mem, yellow);

        Rectangle(mem, ox + 3, 6, ox + 21, 19);



        ox = ICON * 3;

        SelectObject(mem, pen2);

        SelectObject(mem, GetStockObject(NULL_BRUSH));

        Ellipse(mem, ox + 3, 5, ox + 21, 19);



        SelectObject(mem, oldPen);

        SelectObject(mem, oldBrush);

        DeleteObject(pen1); DeleteObject(pen2);

        DeleteObject(yellow); DeleteObject(black);

        SelectObject(mem, oldBmp);
```

```
        DeleteDC(mem);
        ReleaseDC(NULL, scr);

        hImages = ImageList_Create(ICON, ICON, ILC_COLOR24 | ILC_MASK,
COUNT, 0);
        ImageList_AddMasked(hImages, bmp, RGB(255, 0, 255));
        DeleteObject(bmp);
    }

public:
    Toolbar() : hwnd(NULL), hImages(NULL) {}
    ~Toolbar() { if (hImages) ImageList_Destroy(hImages); }

    bool Create(HWND parent, HINSTANCE hInst) {
        INITCOMMONCONTROLSEX icc = { sizeof(icc), ICC_BAR_CLASSES };
        InitCommonControlsEx(&icc);

        hwnd = CreateWindowExW(0, TOOLBARCLASSNAMEW, NULL,
            WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS |
CCS_TOP,
            0, 0, 0, 0, parent, (HMENU)(UINT_PTR)ID_TOOLBAR, hInst,
NULL);
```

```
        if (!hwnd) return false;



        SendMessageW(hwnd, TB_BUTTONSTRUCTSIZE, sizeof(TBBUTTON), 0);



        DrawIcons();

        SendMessageW(hwnd, TB_SETIMAGELIST, 0, (LPARAM)hImages);



        TBBUTTON btn[COUNT] = {};

        const int ids[COUNT] = { IDM_POINT, IDM_LINE, IDM_RECT,
IDM_ELLIPSE };

        for (int i = 0; i < COUNT; i++) {

            btn[i].iBitmap = i;

            btn[i].idCommand = ids[i];

            btn[i].fsState = TBSTATE_ENABLED;

            btn[i].fsStyle = BTNS_CHECKGROUP;

        }

        btn[0].fsState |= TBSTATE_CHECKED;

        SendMessageW(hwnd, TB_ADDBUTTONS, COUNT, (LPARAM)btn);

        return true;

    }
```

```
    void AutoSize() { SendMessageW(hwnd, TB_AUTOSIZE, 0, 0); }

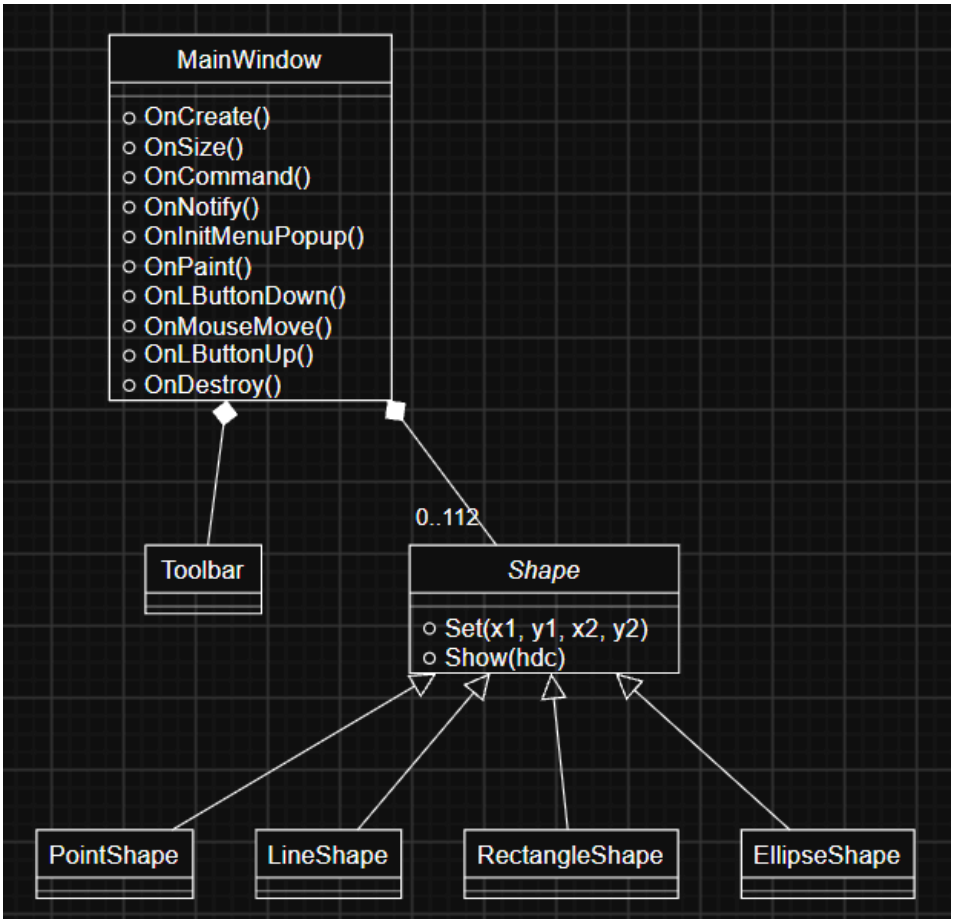
    int Height() const {
        RECT r; GetWindowRect(hwnd, &r);
        return r.bottom - r.top;
    }

    void Check(int id) {
        SendMessageW(hwnd, TB_CHECKBUTTON, id, MAKELONG(TRUE, 0));
    }

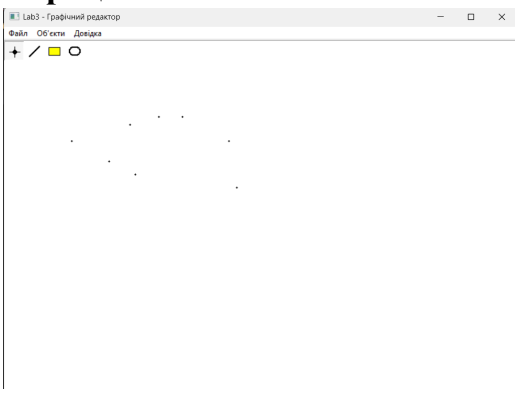
    static const wchar_t* TipText(int id) {
        switch (id) {
        case IDM_POINT:   return L"Крапка";
        case IDM_LINE:    return L"Лінія";
        case IDM_RECT:    return L"Прямокутник";
        case IDM_ELLIPSE: return L"Еліпс";
        }
        return L"";
    }
};
```

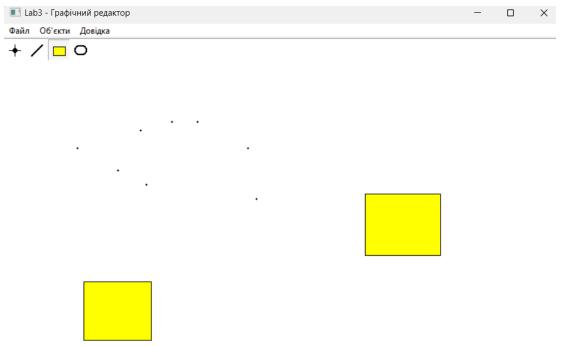
```
#endif
```

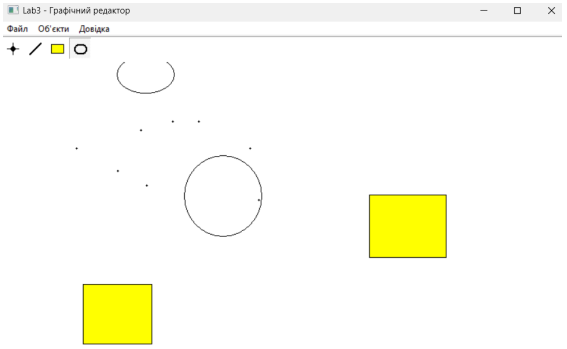
Схеми, діаграми згідно завданню:

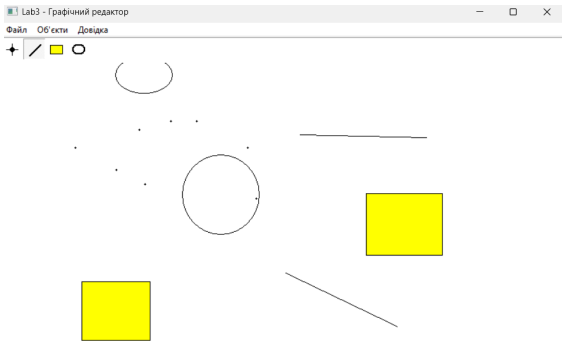


Ілюстрації:









Висновок:

У ході лабораторної роботи №3 я розробила графічний редактор із графічним інтерфейсом користувача на C++ з використанням WinAPI згідно з варіантом Ж = 12. Програма дозволяє вводити крапку, лінію, прямокутник та еліпс, вибираючи тип об'єкта через меню «Об'єкти» або через панель Toolbar з кнопками-іконками та підказками. Застосування абстрактного класу Shape і його нащадків закріпило навички інкапсуляції, успадкування та поліморфізму, а винесення всіх обробників повідомлень, зокрема OnNotify та OnInitMenuPopup, у методи класу MainWindow зробило код структурованішим. Також я відпрацювала роботу з динамічним масивом вказівників на об'єкти, «гумовим» слідом при вводі фігур і синхронізацією кнопок Toolbar з пунктами меню.